Conceptual Architecture of OpenCV

October 8, 2025

SourceArchitects Group Members:

- Sheida Khodabakhshian Khansari
- Vrinda Ashwar
- Matthew Lee
- Werner Bagu-Paiz
- William Moran
- Stefewn Johnson

Introduction to Open Source Computer Vision LIbrary (OpenCV)

- Apache 2 licensed open source software library
- >2500 algorithms for computer vision & machine learning applications such as
  - Face and object recognition
  - Action classification in videos
  - Motion tracking
  - 3D model and point cloud generation
  - Image stitching
  - Image similarity search
  - Eye-tracking
  - Scene recognition
  - Augmented reality integration

- Interfaces for C++, Python, Java, and MATLAB
- Compatible with Windows, Linux, Android, and macOS platforms [1]

Architecture of OpenCV

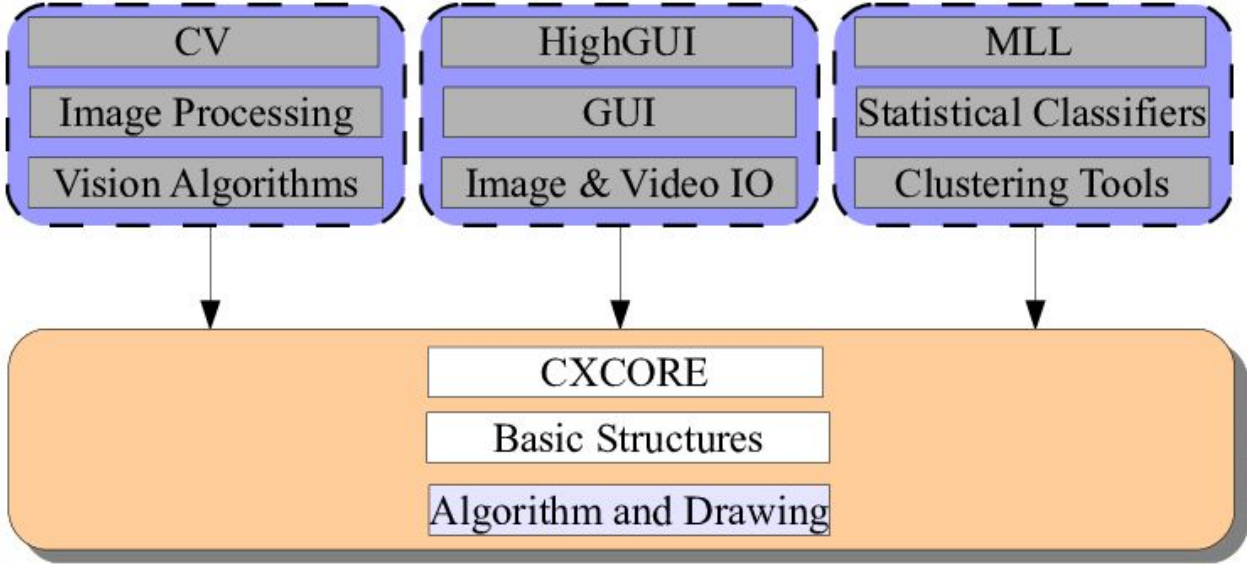


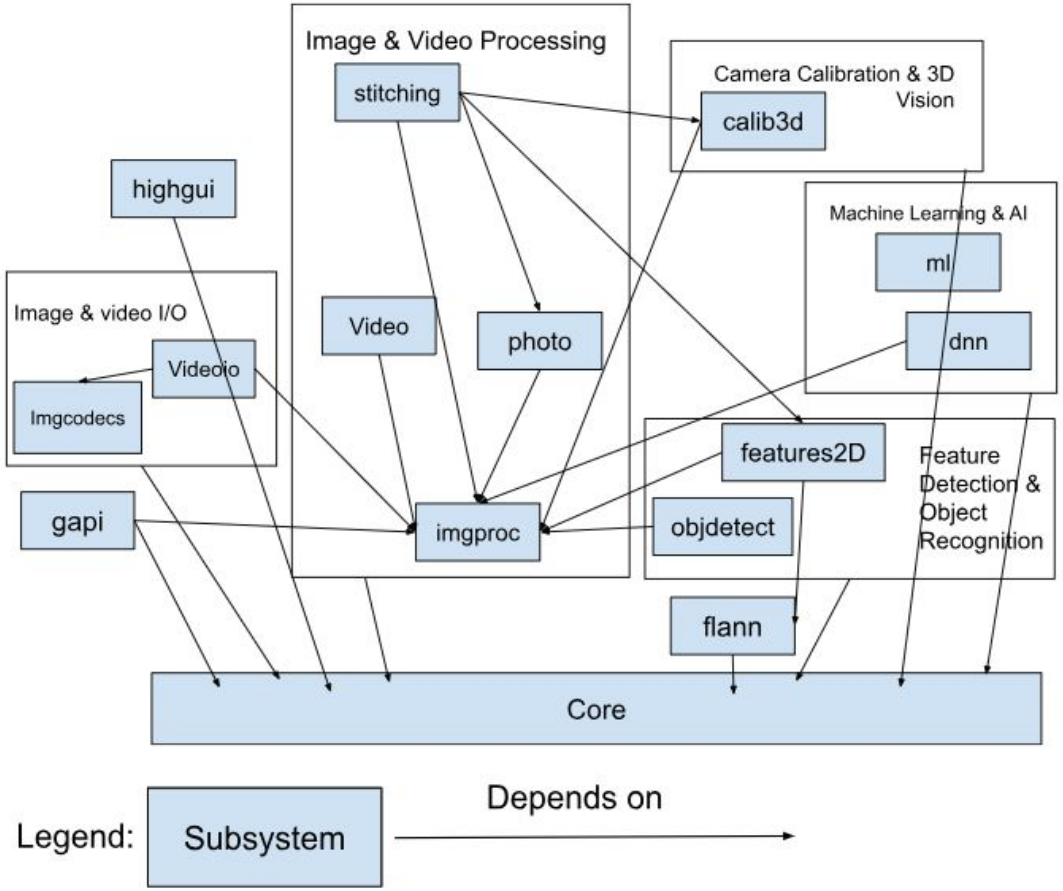
Figure 1. OpenCV Architecture adapted from " OpenCV software architecture.," [2]

Major Functionalities & Associated Modules [3]

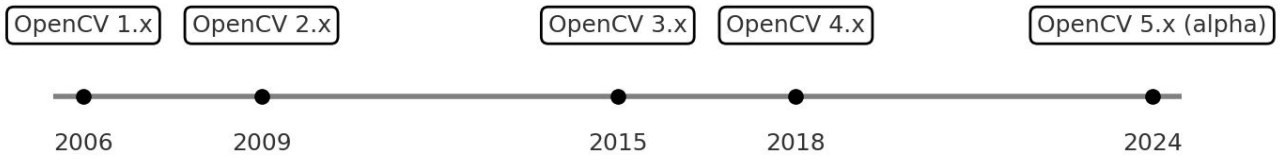
- **Core Functionality:**
  - "Core" : Core Functionality
- **Specialized Modules:**
  - "Gapi" : Graph API
  - "Flann": Clustering & Search in Multi-Dimensional Spaces
- **Image and Video I/O and Display:**
  - "**imgcodecs**": Image file reading & writing
  - "**videoio**" : Video I/O
  - "**highgui**": High-level GUI
- **Image and Video Processing:**
  - "**Imgproc**": Image Processing
  - "**video**" : Video Analysis
  - "**Photo**": Computational Photography
  - "**stitching**" : Images Stitching
- **Camera Calibration and 3D Vision :**
  - "**Calib3d**": Camera Calibration & 3D Reconstruction
- **Feature Detection and Object Recognition:**
  - "**Features2d**": 2D Features Framework
  - "**Objdetect**": Object Detection
- **Machine Learning and AI:**
  - "**dnn**" module : Deep Neural Network
  - "**ml**" : Machine Learning

Conceptual Architecture of OpenCV

Modular Structure

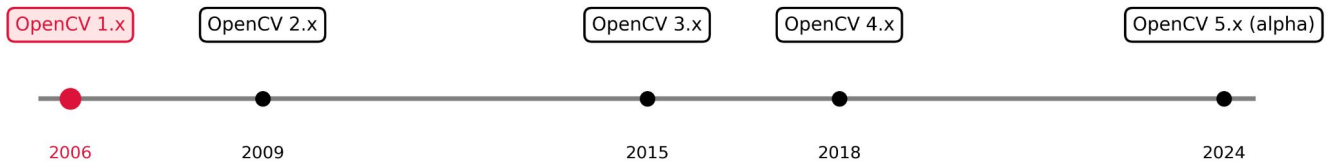


Evolution of the System



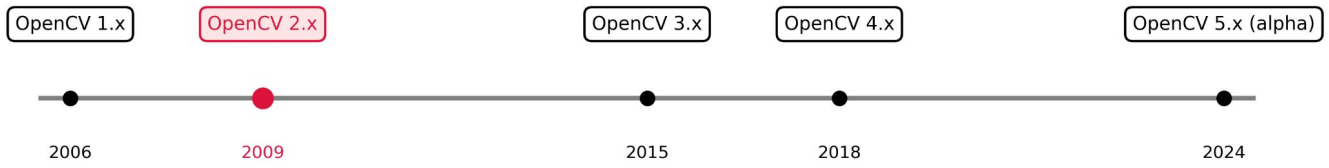
OpenCV 1.x (2006) First Release

- Built mainly in C → limited maintainability [9]
- Early Modules: Core, HighGUI, ML [9]
- Shipped with precompiled Python module [9]
- Support for new image formats (JPEG2000, OpenEXR) [9]



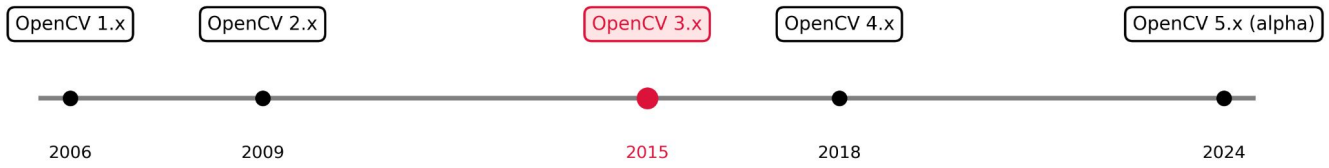
OpenCV 2.x (2009) Architectural Shift

- Introduced C++ interface [9]
- New cv::Mat object → managed memory, improved image handling [9]
- Implementation of CMake → cross-platform builds [9]
- Expanded on machine learning module [9]



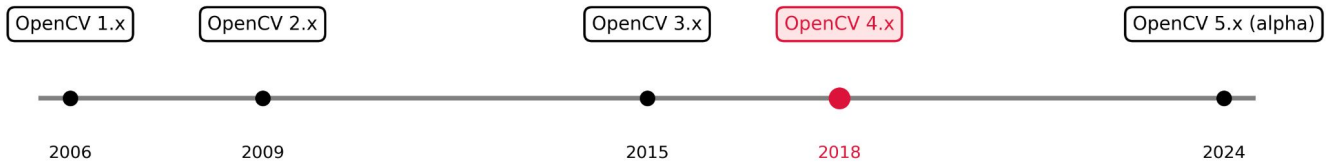
OpenCV 3.x (2015) Further Modernization

- Transparent API (T-API) → automatic GPU acceleration [10]
- Extended Python, Java and Matlab bindings [10]
- Integrated with Intel IPP for optimized functions [10]



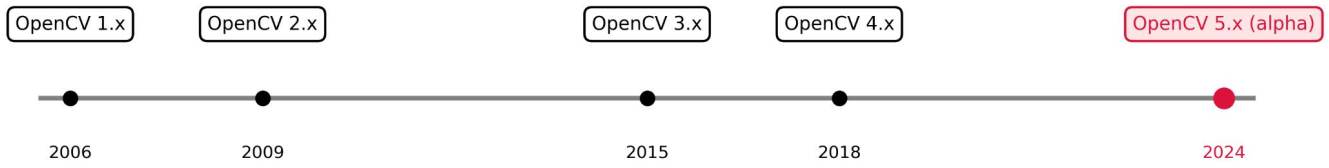
OpenCV 4.x (2018) Full C++ Transition

- Dropped Legacy C API entirely → C++11 [11]
- Added G-API → graph-based pipeline description [11]
- Expanded dnn module → Intel OpenVINO for optimized inference [11]



OpenCV 5.x (2024) Alpha

- Requires C++17 with plans in the future for C++20/23 [11]
- Dropped Python 2 support (Python 3 only) [11]
- Further expansion on dnn module [11]



Data Flow Between OpenCV Components

- OpenCV uses a pipeline-based architecture for image/video processing.
- Data is represented with Mat(C++)/NumPy(Python) for standardization.
- Linear flow with possible branching

Pipeline Structure[8]:

1. Input Acquisition- Images, video streams
2. Preprocessing- Resizing, colour conversion, and filtering
3. Feature Extraction- Edges, key points, descriptors
4. Machine Learning/DNN
5. Output and Visualization- Annotated images, labels, stored results

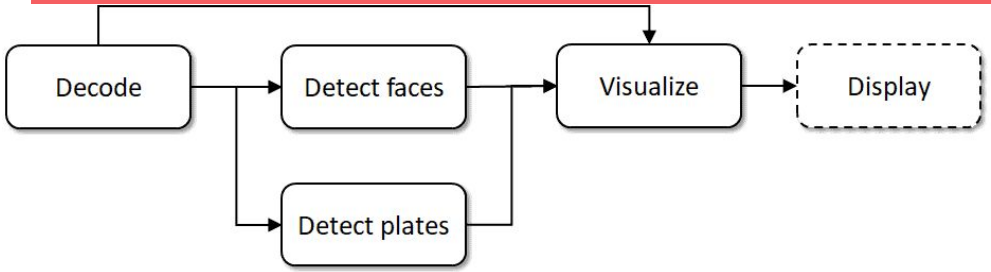


Figure. Privacy Masking Pipeline[8]

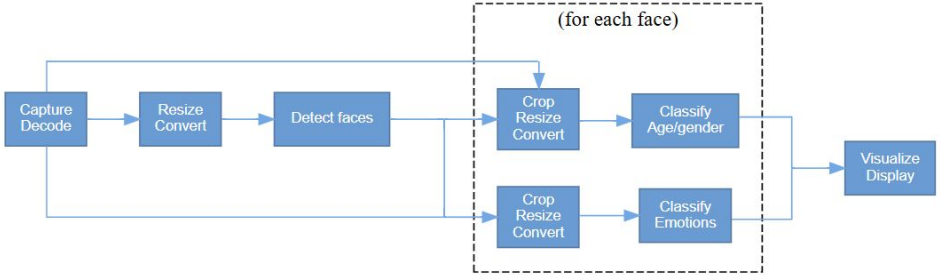
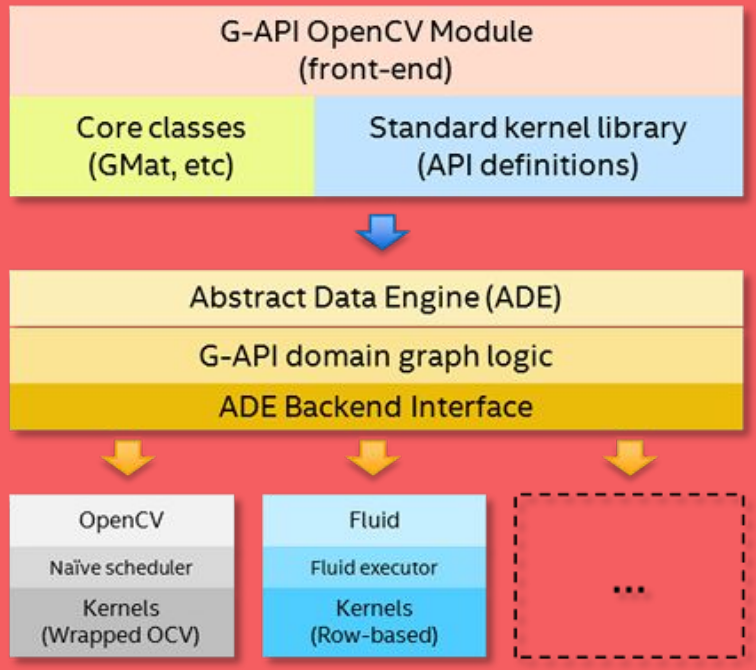


Figure. Interactive Face Detection Pipeline[8]

Control Flow in OpenCV

- Control is user-defined(via Python/C++/Java)
- Often event-driven: frame-by-frame loop in real-time
- Feedback loops exist(e.g. Tracking updates next frame)
- Graph API: defines computation graph, compiled and optimized across backends(CPU/GPU/OpenCL)[6]
- Event-driven + graph-based execution ensures efficiency and concurrency in real-time pipelines.
- Is separated from data to allow flexible coordination of operations.



Layered Architecture

1. API layer
2. Graph Compiler Layer
3. Backends Layer

Concurrency

- OpenCV has built-in native concurrency through parallelization [4]
- Uses cv::parallel_for_ for CPU-intensive operations (e.g., image processing) [4]
- setNumThreads() configures thread count for parallel operations [4]
- If threads = 1 → runs sequentially (no parallelization) [4]
- Execution depends on the backend framework [4]
- Ensures faster, efficient execution of large tasks [4]

Concurrency

- TBB: Uses same thread count unless custom scheduler is made [5]
- OpenMP: No special defined behavior [5]
- Concurrency: Threads = 1 → sequential execution [5]
- GCD: Supports only values ≤ 0 [5]
- C: Special defined behavior [5]
- If no backend, runs sequentially [5]

Division of Responsibilities Among Developers

Communicator
Developer
Tester
User
Integrator

Communicator

- Maksim Shabunin was the main person responsible for this role [7]
- The main duties were to document OpenCV and ensure it was well documented
- The community also played a part in this role [7]
- They would maintain documentation of OpenCV through GitHub

Developer

- Alexander Alekhin, Steve Puttemans, and Maksim Shabunin the main developers [7]
- More than 533 users have also contributed to the development too [7]
- Developers would interact with GitHub workflows
- They write their version of the improved code and submit using pull request
- Report bugs
- Through issues on GitHub, they make extra new requests to add new features

Tester

- No single or group testor responsible for this role
- Developers are the ones who took responsibility [7]
- Developers would create their own test
- This test would be ran by a bot called OpenCV PushBot [7]
- The bot would run automatically only if the pull request was accepted
- The integrator would be the one who accepts the pull request

User

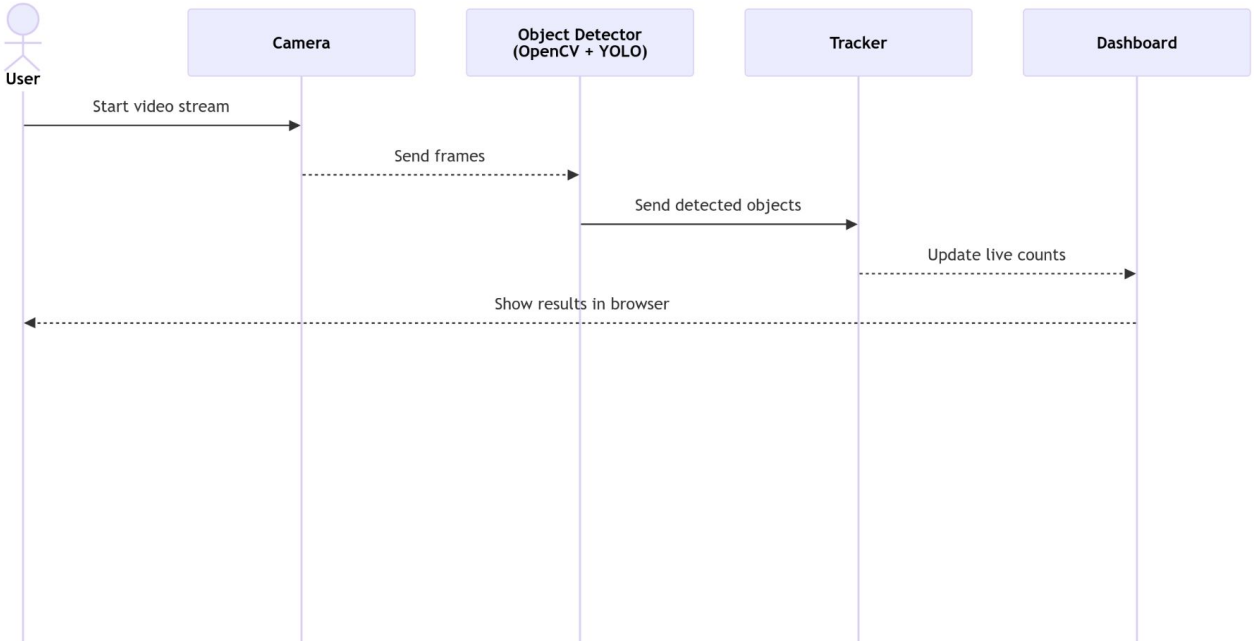
- There are two types of users: Passive users and Active users
- Passive users use OpenCV for their own projects and don’t contribute to OpenCV
- They would download source code or acquire binary distributions
- Active users however do contribute to the development to OpenCV
- They report any issues they find
- Document the code
- Enhance the documentation that already exists
- Some even submit their own code that they wrote themselves

Integrator

- Alexander Alekhin, Steve Puttemans, and Maksim Shabunin the main integrators [7]
- Integrators role is to monitor pull requests on GitHub submitted by developers
- They would review these requests ensuring everything looks good and is proper
- Ensure contributions are of high quality and meet the required criteria
- Complies with the architectural standard set by the project
- Contributions should not bring regressions so they would keep an eye on this too

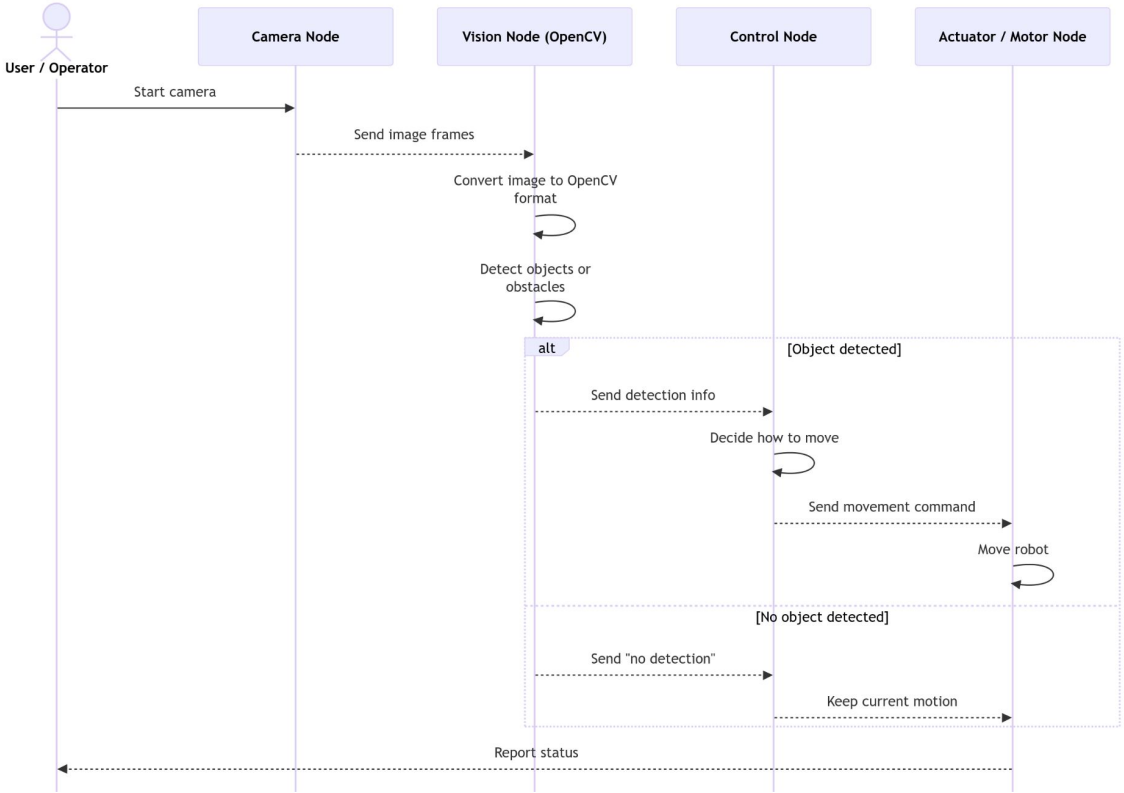
OpenDataCam

- OpenDataCam implements a pipeline architecture for real-time object detection and counting.
- Uses OpenCv and YOLO to detect and count objects [12]



Computer Vision in Robotics

- OpenCV is integrated within a ROS2-based vision system to guide robot movement
- Robots movements are determined by object detection [13]



Conclusion & Lessons Learned

OpenCV is an open sourced computer vision library

- Modular structure
- Pipeline-based architecture
- Layered architecture
- User defined control and event-driven + graph-based execution
- Currently stable release of OpenCV 4.x
- OpenCV 5.x Alpha improves on modernization and better support for features

References:

[1] "About," OpenCV. Accessed: Oct. 01, 2025. [Online]. Available: https://opencv.org/about

[2] "Fig. 10. OpenCV software architecture.," ResearchGate. Accessed: Oct. 06, 2025. [Online]. Available: https://www.researchgate.net/figure/OpenCV-software-architecture_fig17_298799917

[3] "OpenCV: OpenCV modules." Accessed: Oct. 06, 2025. [Online]. Available: https://docs.opencv.org/4.x/index.html

[4] "OpenCV: How to use the OpenCV parallel_for_ to parallelize your code," Opencv.org, 2025. https://docs.opencv.org/4.x/d7/dff/tutorial_how_to_use_OpenCV_parallel_for_.html

[5] "OpenCV: Utility and system functions and macros," Opencv.org, 2025. https://docs.opencv.org/4.x/db/de0/group__core__utils.html#gae78625c3c2aa9e0b83ed31b73c6549c0 (accessed Oct. 07, 2025).

[6] "OpenCV: High-level GUI." Accessed: Oct. 06, 2025. [Online]. Available: https://docs.opencv.org/4.x/d7/dfc/group__highgui.html

[7] "OpenCV: Stakeholders." Accessed: Oct. 07, 2025. [Online]. Available: https://delftswa.gitbooks.io/desosa2016/content/opencv/Chapter.html

[8] Dmitry Matveev, "Hybrid CV/DL pipelines with OpenCV 4.4 G-API," *OpenCV*, Jul. 22, 2020. https://opencv.org/blog/hybrid-cv-dl-pipelines-with-opencv-4-4-g-api/ (accessed Oct. 07, 2025).

[9] "OpenCV Change Logs: v1.0–v2.1." Accessed: Oct. 07, 2025. [Online]. Available: https://github.com/opencv/opencv/wiki/OpenCV-Change-Logs-v1.0%E2%80%90v2.1

[10] "OpenCV 3.0." Accessed: Oct. 07, 2025. [Online]. Available: https://opencv.org/blog/opencv-3-0/

[11] "OpenCV Change Logs: Version 4.12.0." Accessed: Oct. 07, 2025. [Online]. Available: https://github.com/opencv/opencv/wiki/OpenCV-Change-Logs#version4120

[12] "opendatacam / opendatacam," Accessed Oct. 07, 2025. [Online]. Available: https://github.com/opendatacam/opendatacam

[13] I. B. Mansur, "Integrating OpenCV with ROS2" Accessed Oct. 07, 2025. [Online]. Available: https://ibrahimmansur4.medium.com/integrating-opencv-with-ros2-a-comprehensive-guide-to-computer-vision-in-robotics-66b97fa2de92